

Verilog Agent Format

Makarand Bhalerao
University of California, Santa Cruz
Santa Cruz, CA, USA
mbhalera@ucsc.edu

Abstract—Large language models (LLMs) have shown strong performance on code generation tasks, yet their applicability to hardware description language (HDL) optimization remains underexplored. In this work, I investigate supervised fine-tuning (SFT) with low rank adaptation (LoRA) to optimize verbose Verilog code generated by the LiveHD hardware compiler, which produces intermediate representations with many unnecessary GEN wires and redundant constructs. I fine-tune four models namely Qwen2.5-Coder-7B, Qwen3-Coder-30B, DeepSeek-Coder-V2-Lite, and Devstral-Small on a dataset of 4,914 Verilog input-output pairs, evaluating functional correctness using Yosys equivalence checking on a held-out test set of 614 examples. I further introduce an iterative refinement loop where models that produce functionally incorrect outputs are re-prompted with the original Verilog, their failed prediction, and the Yosys error message, enabling self-correction over multiple rounds. I compared training efficiency between SFT and group relative policy optimization (GRPO), analyzing token costs for both approaches. The results show that SFT LoRA effectively learns Verilog cleanup patterns, with larger models achieving higher Yosys pass rates, and iterative refinement further improves correctness on initially failing examples.

Index Terms—Large language models, hardware description languages, supervised fine-tuning, LoRA, Yosys equivalence checking, iterative refinement, GRPO

I. INTRODUCTION

Hardware description languages such as Verilog are fundamental to modern digital circuit design, enabling engineers to specify, simulate, and synthesize complex logic systems. As hardware designs grow in complexity, compiler frameworks like LiveHD have emerged to automate the translation of high-level hardware specifications into Verilog netlists. However, compiler-generated Verilog often suffers from verbosity introducing large numbers of intermediate signals such as GEN wires, redundant assignments, and structurally complex expressions that, while functionally correct, are difficult to read, maintain, and further optimize downstream.

Cleaning and simplifying such compiler output traditionally requires manual effort from experienced hardware engineers or the use of synthesis tools with limited semantic understanding of the original design intent. This creates a bottleneck in hardware development workflows, particularly as the volume of auto-generated HDL code increases.

Recent advances in large language models (LLMs) have demonstrated remarkable capability in understanding and generating code across a wide range of programming languages. Models such as Qwen, DeepSeek-Coder, and Devstral have been pre-trained on large corpora of source code, equipping them with syntactic and semantic knowledge that extends to

hardware description languages. This raises a natural question: *can LLMs be adapted to learn the transformation from verbose compiler-generated Verilog to clean, optimized Verilog?*

In this work, I explore this question by applying supervised fine-tuning (SFT) with low-rank adaptation (LoRA) to four state-of-the-art code LLMs. LoRA enables efficient adaptation of large pre-trained models by injecting trainable low-rank matrices into the attention layers, drastically reducing the number of trainable parameters without sacrificing task performance. I constructed a dataset of 6,143 Verilog input-output pairs derived from LiveHD-generated code and train each model to predict the cleaned output given the verbose input. Functional correctness of model outputs is evaluated using Yosys, an open-source synthesis and formal verification framework, which checks whether the predicted Verilog is logically equivalent to the expected output.

A key challenge in this setting is that models may produce outputs that are syntactically valid but functionally incorrect on the first pass. To address this, I introduced an iterative refinement loop in which failed predictions along with the original Verilog and the Yosys error message are fed back to the model as additional context for self-correction. This mimics a developer’s debug cycle and allows the model to leverage compiler feedback to improve its output.

Finally, I analyzed the training efficiency of SFT relative to group relative policy optimization (GRPO), a reinforcement learning-based fine-tuning method, comparing the number of tokens required under each paradigm for all four models.

II. METHODOLOGY

A. Dataset

The LiveHD Verilog dataset consists of 6,143 examples split into 4,914 training, 615 validation, and 614 test examples. The dataset was constructed through a three-stage pipeline. First, Verilog source files were collected from the MASC group GitHub repository, providing a diverse set of real-world hardware designs. Second, each Verilog file was passed through the Verible formatter to produce clean, consistently formatted Verilog; this formatted output serves as the target (output) side of each training pair, ensuring the model learns to generate well-structured, human-readable Verilog rather than an arbitrary clean style. Third, each original Verilog file was compiled through LiveHD to produce the verbose, compiler-generated representation that serves as the input side of each pair. The resulting LiveHD output contains numerous intermediate `_GEN_` wires, inline source annotations, and

redundant assignments that the model is trained to remove and simplify.

Each example is thus a pair of verbose LiveHD-generated Verilog as input and Verible-formatted clean Verilog as the corresponding output. The dataset is formatted as conversational message pairs with a system prompt, a user message containing the raw Verilog, and an assistant response containing the cleaned output. For DeepSeek and Devstral, a prompt-completion (PC) format is used instead to match each model’s expected input structure during fine-tuning.

B. Models

I fine-tuned four pre-trained code LLMs:

- Qwen2.5-Coder-7B-Instruct - 7B parameter model with 32K context window [1]
- Qwen3-Coder-30B-A3B-Instruct - 30B parameter mixture-of-experts model with 128K context [2]
- DeepSeek-Coder-V2-Lite-Instruct - 16B parameter MoE model with 128K context [3]
- Devstral-Small-2507 24B - Mistral-based code model with 128K context [4]

These models were selected to span a range of sizes and architectures while sharing strong code understanding capabilities.

C. Supervised Finetuning

I applied parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA) [5], which injects trainable low-rank matrices into the attention layers of the frozen pre-trained model, significantly reducing the number of trainable parameters. The LoRA configuration uses rank $r = 16$ and scaling factor $\alpha = 32$, applied to the query, key, value, and output projection layers. Training hyperparameters are as follows: learning rate 2×10^{-5} with cosine decay, per-device batch size of 2 with gradient accumulation over 4 steps (effective batch size 8), and a maximum sequence length of 4096 tokens. Qwen2.5-7B is trained for 3 epochs due to its smaller capacity, while the remaining three models are trained for 1 epoch. All models are trained in bfloat16 precision on a single GPU.

D. Inference

After fine-tuning, each model generates cleaned Verilog for all 614 test examples using greedy decoding (temperature = 0, do_sample=False). Prompts are constructed using each model’s native chat template via `apply_chat_template`, ensuring the input format matches what the model saw during instruction tuning. The system prompt instructs the model to act as a Verilog optimizer that removes redundant `_GEN_` wires and simplifies logic, while the user message contains the raw compiler-generated Verilog.

A key challenge during inference was context window management across models with different limits. Qwen3-30B, DeepSeek-Coder-V2-Lite, and Devstral support 128K token contexts, while Qwen2.5-7B is limited to 32K tokens. Several Verilog examples in the test set, when tokenized with their full prompt, exceeded this limit in some cases reaching

nearly 49K tokens. To handle this, prompts are truncated to `max_ctx - max_new_tokens - 64` tokens before being passed to the model, where the 64-token buffer accounts for special tokens and generation overhead. This truncation was applied only for Qwen2.5-7B, as the other three models context windows were sufficient for all test examples.

All models run inference in bfloat16 precision with `device_map="auto"` for automatic GPU placement. For Qwen3-30B and DeepSeek-Coder-V2-Lite, 4-bit quantization (NF4) via BitsAndBytes is used to fit the models within GPU memory constraints during inference.

Generated outputs undergo a post-processing pipeline before being written to the predictions file. First, markdown code fences (e.g., ````verilog`) are stripped. Then, model-specific special tokens such as `<|im_end|>` and `</s>` are removed. The output is then scanned for the first `module` keyword to discard any preamble text, and truncated after the first `endmodule` to ensure the output contains exactly one well-formed Verilog module. Each prediction is saved alongside its example ID and the ground-truth expected output to a JSONL file, which is then transferred to the Yosys evaluation machine for equivalence checking.

E. Functional Equivalence Evaluation

Generated Verilog outputs are evaluated using Yosys, an open-source hardware synthesis and formal verification framework widely used in digital design workflows. Rather than relying on surface-level metrics such as BLEU score or exact string match, I used logical equivalence checking (LEC) as the evaluation criterion, which directly measures whether the predicted Verilog implements the same Boolean logic as the ground-truth clean Verilog.

Evaluation Pipeline. For each test example, both the model’s predicted Verilog and the ground-truth Verilog are passed to Yosys. Yosys first parses and elaborates each design independently, checking for syntax and elaboration errors. Both designs are then synthesized to an internal gate-level representation using Yosys’s synthesis passes, which normalize the designs into a comparable form by flattening hierarchy, resolving wire assignments, and mapping to a common logic representation. The equivalence check is then performed by comparing the two synthesized netlists on a per-output basis for every output port, Yosys verifies that the combinational logic driving it is identical across both designs under all possible input combinations.

Pass/Fail Criteria. An example is marked as passing only if Yosys confirms full functional equivalence across all outputs. Any of the following conditions result in a failure:

- **Syntax or elaboration errors:** The predicted Verilog cannot be parsed or contains undefined references
- **Functional mismatch:** Yosys finds at least one input assignment for which the predicted and ground-truth outputs differ, reporting a counterexample
- **Structural issues:** Mismatched port lists, missing modules, or incorrect signal widths that prevent the equivalence check from completing

This makes the evaluation strictly binary. A prediction that is syntactically valid but differs in even one output bit under any input condition is counted as a failure.

Error Capture. For each failing example, the Yosys `stdout` and `stderr` streams are captured and stored alongside the example index and ID. These logs include the specific equivalence failure point, counterexample signal values, and any elaboration errors. This structured error output is later used as input to the iterative refinement loop, where the most relevant portion of the failure log identified by keywords such as `FAILURE`, `ERROR`, or `Counterexample` is extracted and injected into the repair prompt to guide the model’s self-correction.

F. Iterative Refinement

A single inference pass is not always sufficient to produce functionally correct Verilog. Models may generate outputs that are syntactically valid but fail Yosys equivalence checking due to subtle logical errors, incorrect wire assignments, or missing signal connections. Rather than accepting these failures, I introduced an iterative refinement round where the model is given a second opportunity to correct its output using structured feedback from the verifier.

Triggering Refinement. After the first-pass Yosys evaluation, all examples where the equivalence check failed are collected into a failure set. Each failure record contains the example ID, return code, and the captured Yosys `stdout` and `stderr` logs which describe the specific nature of the failure. Only failing examples enter the refinement loop passing examples are not re-processed.

Input Preparation. Before the repair inference can run, the failure records must be enriched with the data the model needs to attempt a fix. A preprocessing step merges three sources of information for each failing example:

- 1) The failure record from Yosys (example ID + error logs)
- 2) The original verbose Verilog from the test set (the model’s original input)
- 3) The model’s first-pass prediction (the output that failed)

This produces a unified input record per example containing the original Verilog, the failed prediction, and the Yosys error output, which is then passed to the repair inference pipeline.

Repair Prompt Construction. For each failing example, a repair prompt is dynamically constructed. The prompt includes a system message that instructs the model to act as a Verilog optimizer, followed by a user message that contains:

- The Yosys error log, trimmed to the most relevant failure signal by scanning for keywords such as `FAILURE`, `ERROR`, or `Counterexample` and extracting up to 2000 characters from that point
- The original unformatted Verilog extracted cleanly from the input record
- The previous failed prediction, explicitly labeled so the model knows not to repeat it
- An instruction to fix only the specific issues reported and ensure functional equivalence with the original

This design ensures the model is not retrying blindly each repair prompt is specific to that example’s failure, giving the model precise diagnostic information to guide its correction.

Repair Inference. The same fine-tuned LoRA adapter used for first-pass inference is used for the repair round no separate model is trained for refinement. The same context truncation strategy applies: prompts are capped at `max_ctx - max_new_tokens - 64` tokens to stay within each model’s context window. The same post-processing pipeline (strip fences, extract module block, truncate at `endmodule`) is applied to the repair output before saving.

Scope. One round of iterative refinement (`iter1`) was performed for all four models. A second round (`iter2`) was also attempted; however, the additional round yielded negligible improvement over `iter1`. The examples that remained failing after the first repair pass were consistently re-failed in the second pass, indicating that these represent structurally hard cases that the model cannot self-correct regardless of the number of attempts. This is consistent with the convergence behavior observed in the results, where all functional models plateau at approximately 88% after a single refinement round. Given the diminishing returns, iterative refinement was limited to one round for all models. The repaired predictions from `iter1` are re-evaluated using the same Yosys equivalence checking pipeline and compared against first-pass performance to measure how many previously failing examples were corrected.

G. Token Usage

To compare the computational cost of SFT and GRPO, we measure token consumption for both training and inference.

SFT Training Tokens. Training tokens are counted by tokenizing each training example using the model’s own tokenizer and summing prompt and completion lengths across the entire training set. For Qwen2.5-7B and Qwen3-30B, the full prompt is reconstructed using `apply_chat_template` to match exactly what the model sees during training. For DeepSeek and Devstral, which use a prompt-completion format, both fields are tokenized directly. The total is then multiplied by the number of training epochs for each model.

SFT Inference Tokens. Input tokens are computed by tokenizing the full constructed prompt for each of the 614 test examples. Output tokens are computed by tokenizing each model’s generated prediction. Both are counted using each model’s own tokenizer.

GRPO Training Tokens. GRPO token cost is calculated using the formula:

$$\text{GRPO tokens} = \text{max_steps} \cdot \text{eff_batch} \cdot \text{num_gen} \cdot \text{avg_seq_len} \cdot 3$$

The factor of 3 accounts for the generation pass, policy forward pass, and reference model forward pass per update step. Average sequence length is computed per model by tokenizing the actual GRPO [6] training prompts and adding the maximum completion length of 400 tokens.

III. EVALUATION AND RESULT

A. First Pass Yosys Equivalence results

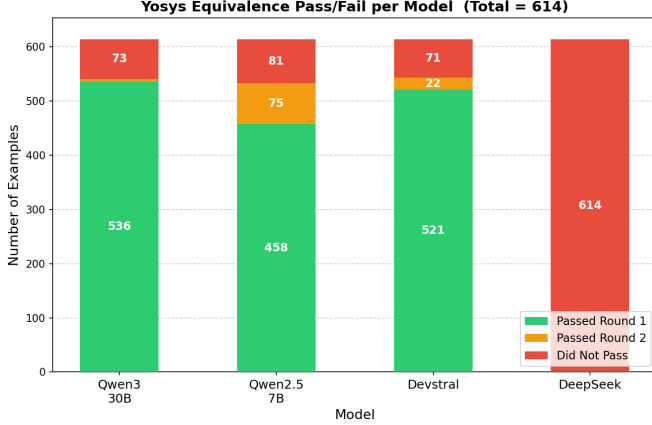


Fig. 1. Yosys Equivalence pass rates for the four models

The table below presents the Yosys logical equivalence check results for all four models on the 614 test examples.

TABLE I
YOSYS LOGICAL EQUIVALENCE CHECK RESULTS ON THE TEST SET

Model	Total	Pass	Fail	Pass Rate
Qwen2.5-7B	614	458	156	74.6%
Qwen3-30B	614	536	78	87.3%
DeepSeek-V2-Lite	614	0	614	0.0%
Devstral	614	521	93	84.8%

Qwen3-30B achieves the highest pass rate among models that produced valid Verilog at 87.3%, with 536 of 614 examples passing equivalence checking. Despite being trained for only 1 epoch, its 30B parameter capacity allows it to reliably learn the Verilog cleanup transformation and generate well-formed, functionally correct output.

Devstral achieves a comparable pass rate of 84.8%, failing on 93 examples, of which 18 were timeouts. The timeout failures indicate that while the generated Verilog was syntactically parseable, the resulting logic was complex enough that Yosys could not complete equivalence verification within the time limit.

Qwen2.5-7B achieves a pass rate of 74.6%, the lowest among models that produced valid Verilog. Notably, 92 of its 156 failures were timeouts rather than outright equivalence failures, suggesting the model frequently generates syntactically valid but structurally complex Verilog that strains the verifier. The lower overall pass rate is attributed to its smaller 7B parameter capacity and its 32K context window, which required prompt truncation for longer examples and may have caused the model to miss critical portions of the input Verilog.

DeepSeek-Coder-V2-Lite failed all 614 Yosys equivalence checks (0% pass rate). Analysis of the failure logs revealed a consistent parse-stage error: "No 'module' definition found in provided Verilog

code." Inspection of the model's predictions showed that DeepSeek generated syntactically collapsed output all whitespace between keywords and identifiers was omitted, producing strings such as

```
modulecounter_184(inputclk_9040,
outputreg[31:0]count_7176)
```

instead of valid space-separated declarations. The Verilog content itself is logically correct the circuit structure and semantics are preserved however, Yosys's parser requires keyword-identifier separation to tokenize a valid module declaration. Since no parseable module block could be found in any prediction, equivalence checking could not proceed for any example. This failure is attributed to a formatting artifact in the model's output rather than a failure of logical understanding, and notably a post-processing step to normalize whitespace could potentially recover these predictions without any retraining.

B. Second Pass Yosys Equivalence results

TABLE II
YOSYS RESULTS WITH ITERATIVE REFINEMENT

Model	Pass 1	Pass 2	Total	Final Rate
Qwen2.5-7B	458	75	533	86.8%
Qwen3-30B	536	5	541	88.1%
DeepSeek-V2-Lite	0	0	0	0.0%
Devstral	521	22	543	88.4%

The most striking observation is that despite significant differences in Round 1 pass rates, all three models that produced valid Verilog converge to a narrow overall pass rate band of 86.8–88.4% after iterative refinement. This convergence suggests a hard ceiling of approximately 88% on this test set for these model families under single-round repair, beyond which the remaining failures represent structurally harder examples that the models consistently fail to correct.

Qwen2.5-7B achieves the highest recovery rate at 48.1%, recovering 75 of its 156 first-pass failures. This is notable because Qwen2.5-7B had the lowest Round 1 pass rate, yet nearly half its initial failures were correctable on a second attempt. This suggests that many of its Round 1 failures were marginal cases examples where the model generated output close to correct but with minor errors that the structured Yosys feedback was sufficient to fix rather than fundamental limitations in the model's understanding of the transformation.

Qwen3-30B, despite achieving the best Round 1 performance, has the lowest recovery rate at 6.4%, recovering only 5 of its 78 failures. This indicates that the examples Qwen3-30B fails in Round 1 are consistently difficult cases that the model cannot self-correct even with verifier feedback. Its failures are more likely to be structurally complex Verilog patterns that fall outside what the model learned during fine-tuning.

Devstral recovers 22 of its 93 failures (23.7%), a moderate recovery rate that reflects a mix of marginal and hard failure cases. Its overall pass rate of 88.4% is the highest among all

models after refinement, marginally edging out Qwen3-30B at 88.1%.

DeepSeek-Coder-V2-Lite achieves no recovery in Round 2 (0%). As discussed previously, its failure mode is a systematic whitespace formatting issue that is deterministic the model consistently collapses token boundaries regardless of the repair prompt. Since the same underlying generation behavior persists in the second pass, providing Yosys error feedback does not help, and the repair loop produces identically malformed output. Recovery for DeepSeek would require either a post-processing normalization step or retraining with explicit formatting constraints.

C. Failure Analysis

This section examines the examples that failed Yosys equivalence checking in Round 1, which of those were recovered in the second pass (Iter-1), and which never passed across either attempt. Failures fall into three categories: *Not Equivalent* (the checker formally proves a logical difference exists), *Timeout* (the SAT solver exceeded the 30-second limit before reaching a verdict), and *Parse Error* (the output could not be parsed as valid Verilog at all).

Qwen3 30B

Of the 78 Round 1 failures, only 5 were recovered in Iter-1 (6.4% recovery rate), leaving 73 examples that never passed. Of those 73 persistent failures, 69 were formally proved not equivalent and 4 fell into other error categories. The low recovery rate indicates that Qwen3 30B’s failures are largely deterministic when the model produces a logically incorrect output it tends to reproduce the same mistake on a second attempt.

The dominant pattern across the persistent failures is the removal of the `signed` keyword from port and signal declarations. The example below shows `memory_sync_216.v`, which failed both rounds:

Expected:

```
module memory_sync_216 (
    input  clk_1148,
    input  we_8917,
    input  [4:0] addr_6429,
    input  signed [31:0] din_6260,
    output reg signed [31:0] dout_2377
);
    reg signed [31:0] mem [0:31];
    always @(posedge clk_1148) begin
        if (we_8917)
            mem[addr_6429] <= din_6260;
            dout_2377 <= mem[addr_6429];
    end
endmodule
```

Prediction:

```
module memory_sync_216 (
    input  clk_1148,
    input  we_8917,
    input  [4:0] addr_6429,
    input  [31:0] din_6260,
    output [31:0] dout_2377
);
    reg [31:0] mem [0:31];
```

```
always @(posedge clk_1148) begin
    if (we_8917)
        mem[addr_6429] <= din_6260;
        dout_2377 <= mem[addr_6429];
    end
endmodule
```

The model strips `signed` from all declarations and also removes `reg` from `dout_2377`, changing it from a register to a wire. Both changes alter the circuit’s arithmetic behavior and assignment semantics, causing the equivalence check to fail. The model makes these same mistakes consistently across both inference attempts, which is why no second pass can recover these examples.

Qwen2.5 7B

Of the 156 Round 1 failures, 92 were timeouts and 60 were not equivalent. This split is significant: all 75 examples recovered in Iter-1 came exclusively from the timeout category not a single not-equivalent failure was corrected. This reveals that timeouts are caused by solver complexity rather than model error. When the generated code is structurally complex, the SAT solver may fail to reach a decision within 30 seconds even if the output is logically correct. A second inference attempt can produce a structurally simpler but equivalent output that the solver can verify within the limit.

The following shows a recovered example `counter_en_89.v` timed out in Round 1 but passed identically in Iter-1:

```
module counter_en_89 (
    input  clk_6163,
    input  rst_4651,
    input  en_8360,
    output reg [9:0] cnt_5336
);
    always @(posedge clk_6163) begin
        if (rst_4651)
            cnt_5336 <= 10'd0;
        else if (en_8360)
            cnt_5336 <= cnt_5336 + 1;
    end
endmodule
```

The prediction is token-for-token identical to the expected output, confirming that the Round 1 failure was entirely a solver timeout and not a model mistake.

For the 81 never-passed examples, the dominant circuit types are `memory_sync` (26 examples), `structural_chain` (18), `register_file` (16), `memory_be` (10), and `mealy_fsm` (8) all structurally complex stateful circuits. The same `signed` keyword omission appears in the never-passed memory examples:

Prediction for memory_sync_216.v:

```
module memory_sync_216 (
    input  clk_1148,
    input  we_8917,
    input  [4:0] addr_6429,
    input  [31:0] din_6260,
    output reg [31:0] dout_2377
);
    reg [31:0] mem [0:31];
    always @(posedge clk_1148) begin
        if (we_8917)
```

```

        mem[addr_6429] <= din_6260;
        dout_2377 <= mem[addr_6429];
    end
endmodule

```

For FSM circuits the output is frequently truncated mid-generation, suggesting the model hits a token limit on complex multi-state machines and produces incomplete Verilog that cannot be parsed or verified.

Devstral

Of the 93 Round 1 failures, 71 were not equivalent and 18 were timeouts. Of the 22 recovered in Iter-1, 16 came from timeouts and 6 from not-equivalent cases. The recovery of 6 not-equivalent failures distinguishes Devstral from Qwen3 and suggests a small degree of non-determinism in its outputs on a second attempt it occasionally generates a different and correct solution.

The 71 persistent failures show the same signed omission on `memory_sync_216.v`:

Prediction:

```

module memory_sync_216 (
    input  clk_1148,
    input  we_8917,
    input  [4:0] addr_6429,
    input  [31:0] din_6260,
    output reg [31:0] dout_2377
);
    reg [31:0] mem [0:31];
    always @(posedge clk_1148) begin
        if (we_8917)
            mem[addr_6429] <= din_6260;
            dout_2377 <= mem[addr_6429];
    end
endmodule

```

Notably, `memory_sync_216.v` appears in the never-passed list of all three working models Qwen3 30B, Qwen2.5 7B, and Devstral. This is not a coincidence: examining the never-passed example IDs reveals a shared core set of hard examples (`ex_6`, `ex_25`, `ex_50`, `ex_51`, `ex_55`, `ex_62`, `ex_68`) that fail across all models in both rounds. This strongly suggests these are genuinely difficult circuits in the test set that no current fine-tuned model handles correctly, rather than weaknesses specific to any one model. The signed qualifier appears to be the most consistent source of these shared failures all three models systematically drop it during simplification, and no second inference attempt corrects this behavior.

DeepSeek-Coder 16B

All 614 examples failed both rounds due to the whitespace collapse issue described in the anomaly section. The following shows `counter_184.v`:

Expected:

```

module counter_184 (
    input  clk_9040,
    input  rst_8511,
    output reg [31:0] count_7176
);
    always @(posedge clk_9040) begin
        if (rst_8511)

```

```

        count_7176 <= 32'd0;
    else
        count_7176 <= count_7176 + 32'd1;
    end
endmodule

```

Prediction:

```

modulecounter_184(inputclk_9040,inputrst_8511,
outputreg[31:0]count_7176);always@(posedgeclk_9040)
beginif(rst_8511)count_7176<=32'd0;elsecount_7176<=
count_7176+32'd1;endendmodule

```

The logic is correct but all whitespace between keywords and identifiers is removed, making the output unparseable by Yosys. This pattern is consistent across all 614 examples in both rounds the formatting failure is deterministic and a second inference pass reproduces the same collapsed output.

Overall Patterns

Three consistent patterns emerge across all models. First, timeout failures are recoverable the large majority of Round 1 timeouts passed on the second attempt across both Qwen2.5 7B and Devstral, indicating that solver complexity is sensitive to the specific structure of the generated code and a simpler reformulation can succeed. Second, not-equivalent failures are persistent once the checker formally proves a logic difference, a second inference attempt almost never corrects it, pointing to systematic model behavior rather than random generation noise. Third, the signed keyword is a shared blind spot all three working models consistently drop signed qualifiers from memory and arithmetic circuits, and this single omission accounts for a significant portion of the shared persistent failures across the entire test set.

D. Token Usage Analysis

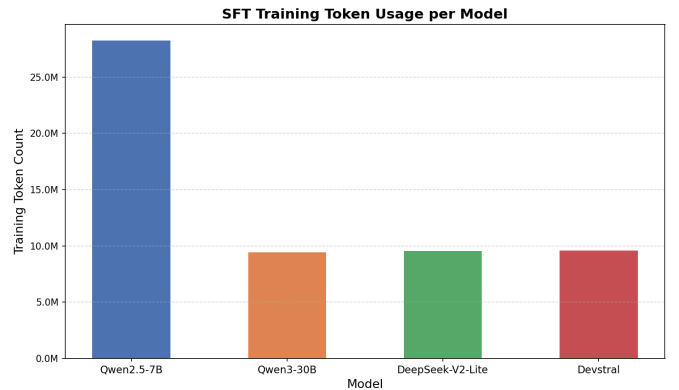


Fig. 2. Tokens used by each model while training

Training tokens. Qwen2.5-7B consumes significantly more training tokens (28.3M) than the other three models, which all fall in the 9.4–9.6M range. This difference is attributable to the number of training epochs Qwen2.5-7B was trained for 3 epochs while the remaining models were trained for 1 epoch. The larger models (Qwen3-30B, DeepSeek-Coder-V2-Lite, and Devstral) were trained for a single epoch because their

greater parameter count and extensive pre-training on code corpora provided sufficient Verilog knowledge to adapt quickly to the cleanup task with minimal fine-tuning passes. Qwen2.5-7B, being a smaller model with more limited capacity, required additional epochs to adequately learn the Verilog transformation patterns from the training data. When normalized to a single epoch, all four models consume approximately 9.4–9.6M tokens, reflecting the fact that they all train on the same underlying dataset with similar tokenization behavior across model families.

Inference tokens. As shown in Figure 3, inference token usage is remarkably consistent across all four models. Input tokens are 876K for both Qwen2.5-7B and Qwen3-30B, and slightly higher at 897K and 894K for DeepSeek-V2-Lite and Devstral respectively. Output tokens are similarly close, ranging from 96K (DeepSeek-V2-Lite) to 106K (Devstral), with both Qwen models generating 104K output tokens.

The near-identical input token counts confirm that inference cost is driven entirely by the test set rather than by the model or its training configuration. All four models are evaluated on the same 614 examples with similarly structured prompts, so the underlying Verilog content is identical across all runs. The small differences in input tokens between the Qwen models (876K) and DeepSeek/Devstral (894–897K) reflect minor differences in how each model’s chat template encodes the system prompt and user message rather than any difference in input content. The dominance of input tokens over output tokens roughly an 8:1 ratio across all models is expected given that the input Verilog prompts are substantially longer than the cleaned output the model generates. Output token variation across models is minimal and reflects slight differences in generation verbosity rather than task complexity.

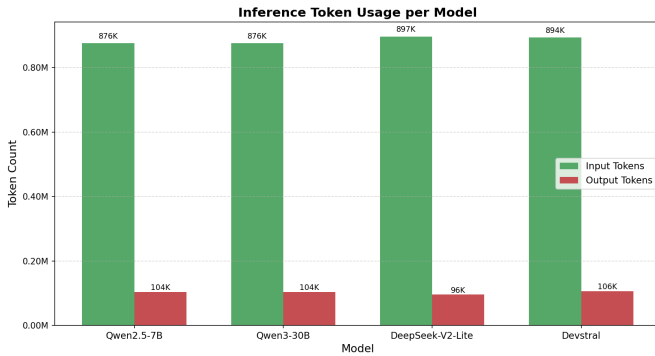


Fig. 3. Tokens used by each model during inference

As shown in Figure 4, the SFT vs GRPO training token comparison reveals a clear difference in token efficiency depending on the number of training epochs. For Qwen3-30B, DeepSeek-V2-Lite, and Devstral all trained for a single SFT epoch the SFT and estimated GRPO training token counts are broadly comparable, with SFT at 9.4–9.6M tokens and GRPO estimates at 10.3–10.5M tokens. The slight edge of GRPO over single-epoch SFT in this range is due to the

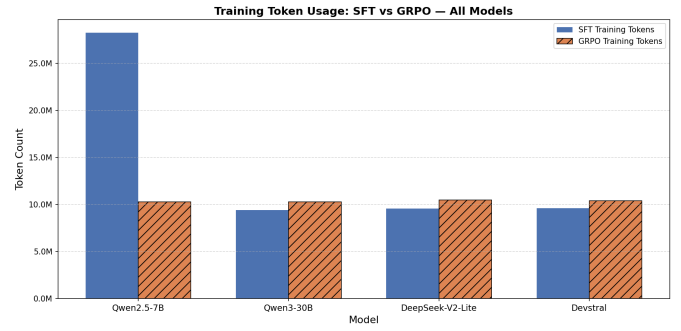


Fig. 4. Tokens used by each model during GRPO vs SFT

three forward passes required per GRPO update step, which marginally increases token consumption relative to the single forward-backward pass of SFT.

The most striking difference appears for Qwen2.5-7B, where SFT training tokens reach 28.3M due to the three training epochs required for the smaller model to converge. In contrast, the estimated GRPO token cost for Qwen2.5-7B is only 10.3M a 2.7× reduction. This gap highlights a key practical advantage of GRPO for smaller models: since GRPO’s token cost is governed by a fixed configuration (number of steps, batch size, and generations) rather than dataset size and epochs, it does not scale with the number of passes over the data. For models that require multiple SFT epochs to achieve adequate performance, GRPO offers a significantly more token-efficient training alternative, provided a suitable reward signal such as Yosys equivalence checking can be integrated into the training loop.

E. Similarity metrics

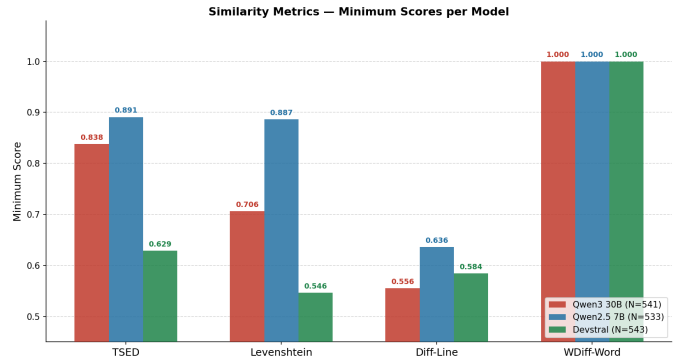


Fig. 5. Similarity metrics for examples passing Yosys equivalence

Similarity metrics were computed exclusively on predictions that passed Yosys equivalence checking, since measuring textual closeness is only meaningful when logical correctness is already confirmed. DeepSeek-R1 is excluded from this analysis as it produced zero passing examples.

Four metrics were used to evaluate how closely each model’s prediction matches the expected clean Verilog output. TSED (Tree Structural Edit Distance) measures structural

similarity at the abstract syntax tree level, capturing how much the parse tree of the prediction differs from the reference. Levenshtein similarity is a character-level edit distance normalized by the length of the longer string, penalizing any character-level deviation. Diff-Line is a line-based similarity derived from unified diff output, reflecting how many lines differ between prediction and reference. WDiff-Word is a word-level similarity that measures how many individual tokens, keywords, identifiers, operators were changed between the two outputs.

All three models achieve near-perfect similarity scores across all metrics. The mean and maximum scores are 1.0 for every model and every metric, meaning the typical passing prediction is token-for-token identical to the reference. More than 75% of passing examples score exactly 1.0 on every metric, meaning the model reproduces the expected clean Verilog verbatim in the vast majority of cases. Since max and mean collapse to 1.0 across the board, Figure X reports only the minimum scores per model per metric, which represent the worst-case outputs that still passed Yosys equivalence checking.

WDiff-Word shows a minimum of 1.000 for all three models, meaning that not a single passing example had any word-level difference from the reference. This indicates that the models never introduce or substitute tokens when their output is logically correct the vocabulary of the predicted Verilog is always identical to the expected output at the word level.

For TSED and Levenshtein, Qwen2.5 7B has the highest minimum scores at 0.891 and 0.887 respectively, meaning even its worst-case passing outputs are structurally and character-wise close to the reference. Qwen3 30B follows with minimums of 0.838 on TSED and 0.706 on Levenshtein, while Devstral shows the lowest minimums on both metrics at 0.629 and 0.546, suggesting that its rare outlier cases deviate more significantly from the reference even when logically equivalent.

The Diff-Line metric shows a different ordering Qwen3 30B has the lowest minimum at 0.556, slightly below Devstral at 0.584, while Qwen2.5 7B again shows the highest minimum at 0.636. This reversal suggests that Qwen3 30B occasionally produces outputs that differ across more lines than expected, possibly due to different formatting or line-breaking choices, even though its character-level and structural similarity remains reasonable.

Overall, the minimum scores across all metrics and models remain above 0.546, confirming that even the worst-case passing predictions maintain a meaningful degree of similarity to the reference clean Verilog output. These results indicate that when the models produce logically correct outputs, they also tend to produce outputs that are stylistically very close to the ground truth, suggesting the fine-tuning has successfully captured both the structural and syntactic patterns of clean Verilog-2001.

F. Discussion

Model Size Does Not Guarantee Success. The results challenge the assumption that larger models always outperform smaller ones on domain-specific tasks. DeepSeek-Coder-V2-Lite, despite being a 16B parameter model, failed entirely due to a systematic output formatting issue, while Qwen2.5-7B at 7B parameters achieved a 74.6% first-pass rate and 86.8% after refinement. This highlights that instruction-following reliability and output formatting are as critical as model capacity for tasks with strict syntactic requirements like Verilog generation. A model that generates logically correct but unparseable output provides no practical value in a hardware verification pipeline.

Convergence to a Hard Ceiling. A key finding is the convergence of all three functional models to a narrow pass rate band of 86.8–88.4% after one round of iterative refinement, despite starting from significantly different Round 1 pass rates (74.6% to 87.3%). This convergence suggests that approximately 11–13% of the test set represents structurally hard examples that are consistently difficult for all models regardless of size or architecture. These are likely Verilog modules with complex combinational logic, deep signal dependencies, or unusual LiveHD-specific patterns that were underrepresented in the training data.

Iterative Refinement is Most Effective for Weaker Models. The recovery rate analysis reveals an inverse relationship between Round 1 performance and refinement gain Qwen2.5-7B had the lowest first-pass rate but the highest recovery rate (48.1%), while Qwen3-30B had the best first-pass rate but the lowest recovery (6.4%). This suggests that stronger models fail on genuinely hard examples that cannot be self-corrected, while weaker models fail on marginal cases where structured Yosys feedback is sufficient to guide correction. This has a practical implication: iterative refinement adds the most value when paired with a model that has moderate but not exceptional base performance.

Timeout Failures Indicate Generated Complexity. A significant portion of failures across Qwen2.5-7B (92 of 156) and Devstral (18 of 93) were Yosys timeouts rather than equivalence failures. This means the generated Verilog was syntactically valid and parseable, but the logic was too complex for the verifier to check within the time limit. This points to a secondary failure mode distinct from logical incorrectness the models sometimes over-engineer the output, generating unnecessarily complex circuit structures that are harder to verify than the ground truth.

Context Window as a Bottleneck. Qwen2.5-7B’s 32K context window required prompt truncation for longer Verilog examples, which likely contributed to its lower first-pass performance. Larger models with 128K context windows could accommodate the full input without truncation, giving them a structural advantage on complex, long-form Verilog inputs. This underscores the importance of context window size as

a practical consideration when selecting models for hardware code tasks, where input lengths can be substantial.

CONCLUSION

This work investigated the application of supervised fine-tuning with Low-Rank Adaptation (LoRA) to automate the optimization of verbose compiler-generated Verilog produced by the LiveHD hardware compiler. Four state-of-the-art code language models Qwen2.5-Coder-7B, Qwen3-Coder-30B, DeepSeek-Coder-V2-Lite, and Devstral-Small were fine-tuned on a dataset of approximately 5,000 Verilog input-output pairs and evaluated using Yosys logical equivalence checking on a held-out test set of 614 examples. An iterative repair loop was further introduced to recover examples that failed first-pass verification by re-prompting each model with structured Yosys error feedback.

The first-pass results demonstrated that three of the four models successfully learned the Verilog cleanup transformation, achieving pass rates of 74.6% (Qwen2.5-7B), 87.3% (Qwen3-30B), and 84.8% (Devstral). DeepSeek-Coder-V2-Lite failed entirely due to a systematic output formatting issue where whitespace between keywords and identifiers was collapsed during generation, producing unparseable Verilog despite logically correct content. This highlights that instruction-following reliability and output formatting are as critical as model capacity in hardware verification pipelines where strict syntactic correctness is required.

After one round of iterative refinement, all three functional models converged to a narrow pass rate band of 86.8–88.4%, despite starting from significantly different Round 1 baselines. This convergence points to a hard ceiling on this test set at approximately 88% under single-round repair, with the remaining failures representing structurally complex examples that models consistently cannot self-correct. Notably, Qwen2.5-7B achieved the highest recovery rate of 48.1% despite having the lowest first-pass rate, suggesting that smaller models fail more on marginal cases that are amenable to correction, while larger models fail on genuinely harder examples. Iterative refinement was thus most effective when applied to models with moderate base performance.

The token usage analysis revealed that for single-epoch SFT, training costs are broadly comparable to estimated GRPO costs (9.4–9.6M vs. 10.3–10.5M tokens). However, for smaller models requiring multiple training epochs, such as Qwen2.5-7B at 3 epochs (28.3M tokens), GRPO offers a significantly more token-efficient alternative at an estimated 10.3M tokens a $2.7\times$ reduction provided a suitable reward signal such as Yosys equivalence checking can be integrated into the training loop.

Overall, these results demonstrate that parameter-efficient fine-tuning of code LLMs is a viable and practical approach for automating HDL code optimization, achieving near 90% functional equivalence on compiler-generated Verilog with a relatively small training dataset and modest compute. The combination of SFT LoRA fine-tuning and Yosys-guided iterative refinement provides a strong baseline for LLM-assisted

hardware design workflows, with clear directions for further improvement through multi-round refinement, post-processing normalization, and reinforcement learning with formal verification rewards.

REFERENCES

- [1] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, “Qwen2.5-coder technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.12186>
- [2] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, “Qwen3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [3] DeepSeek-AI, Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma, W. Zeng, X. Bi, Z. Gu, H. Xu, D. Dai, K. Dong, L. Zhang, Y. Piao, Z. Gou, Z. Xie, Z. Hao, B. Wang, J. Song, D. Chen, X. Xie, K. Guan, Y. You, A. Liu, Q. Du, W. Gao, X. Lu, Q. Chen, Y. Wang, C. Deng, J. Li, C. Zhao, C. Ruan, F. Luo, and W. Liang, “Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.11931>
- [4] A. Rastogi, A. Yang, A. Q. Jiang, A. H. Liu, A. Sablayrolles, A. Héliou, A. Martin, A. Agarwal, A. Ehrenberg, A. Lo, A. Roux, A. Darce, A. Mensch, B. Bout, B. Rozière, B. D. Monicault, C. Bamford, C. Wallenwein, C. Renaudin, C. Lanfranchi, C. Denoix, C. Barreau, D. D. Mizelle, D. de las Casas, E. Chane-Sane, E. Fugier, E. B. Hanna, G. Berrada, G. Delerce, G. Guinet, G. Novikov, G. Neubig, G. Lample, G. Martin, H. Jaju, J. Ludziewski, J. Rute, J.-M. Delignon, J. Chabran, J. Studnia, J. Barmentlo, J. Amar, J. S. Roberts, J. Denize, K. Saxena, K. Yadav, K. Khandelwal, K. R. Chandu, K. Jain, L. R. Lavaud, L. Blier, L. Zhao, L. Martin, L. Saulnier, L. Gao, M. Pellat, M. Guillaumin, M. Felardos, M. Dinot, M. Darrin, M. Augustin, M. Seznec, N. Gupta, N. Raghuraman, O. Duchenne, P. Wang, P. von Platen, P. Saffer, P. Jacob, P. Wambergue, P. Kurylowicz, P. Chagniot, P. Stock, P. Agrawal, R. Delacourt, R. Soletskyi, R. Sauvestre, S. Vaze, S. Gandhi, S. Subramanian, S. Dalal, S. Gandhi, S. Ghosh, S. Mishra, S. Aithal, S. Antoniak, T. L. Scao, T. Lavril, T. Schueller, T. Foubert, T. Robert, T. Wang, T. Lacroix, T. Bewley, V. Nemychnikova, V. Paltz, V. Richard, W.-D. Li, W. Marshall, X. Wang, X. Zhang, Y. Wan, and Y. Tang, “Devstral: Fine-tuning language models for coding agent applications,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.25193>
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [6] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.03300>